SENDA

Stellar social finance + creator commerce + freelance marketplace

FINAL PRODUCT SPECIFICATION | CLAUDE IMPLEMENTATION REFERENCE | 24 OCT 2026

Scope: a real mobile-responsive Stellar testnet application with a clean migration path to mainnet, not a clickable mockup. This document defines product behavior and architecture; integrations and deployments must be validated and executed by the implementing engineer.

01 / Product direction

Senda is a social-first wallet and marketplace. Users sign up by email, obtain a Stellar-compatible account, send or receive XLM/USDC, share and monetize posts, tip creators, hire freelancers, deliver work through chat and settle jobs through Soroban escrow. A separate opt-in USDC Auto Earn experience may connect to a real lending or vault strategy.

Design promise: social-first simplicity; blockchain complexity hidden but never transactions, fees, risks or user consent. Core on-chain transactions must have real Stellar hashes and authoritative confirmation.

02 / Launch mode and funding truth

• Default release is Stellar TESTNET: live network transactions with valueless test assets, not fake application balances.

• Onboarding: fund freshly generated Stellar address with test XLM, create the correct USDC trustline/asset capability, then transfer supported TEST USDC. The official testnet USDC availability and issuer must be verified, never assumed.

• Faucet: first signup funding plus user-triggered additional claims; per-identity/device/IP throttling, quotas, abuse monitoring, treasury balance checks, and idempotent claims.

• Mainnet: XLM/USDC rewards require a genuinely funded treasury, risk controls, legal review, and anti-Sybil protections; never promise an unlimited real-money faucet.

• Real USDC issuance and Stellar reserves/trustlines must be verified per selected network.

03 / Paiflow workflows and responsibilities

Paiflow is a visual Soroban workflow builder with documented On Receive, Schedule, HTTP Webhook, Pay, Split, Swap and Condition building blocks. Paiflow documentation describes a testnet deployment and prepare/sign/submit flows; inspect current SDK and API before coding. “Workflow IDs” below are proposed Senda configuration names, not guaranteed Paiflow backend types.

Workflow / key	Triggered by	Expected handling
SEND / PAIFLOW_SEND_WORKFLOW_ID	User confirms XLM/USDC transfer	Prepare/sign/submit an authorized payment; if Paiflow unsupported, direct Stellar transfer
RECEIVE / PAIFLOW_RECEIVE_WORKFLOW_ID	Incoming payment or receive trigger	Match receiving wallet and transaction; ledger confirmation and idempotent webhook

Workflow / key	Triggered by	Expected handling
TIP / PAIFLOW_TIP_WORKFLOW_ID	Tip post / profile	User authorized send, optional split to creator and fee collector
POST_PURCHASE / PAIFLOW_POST_PURCHASE_WORKFLOW_ID	Buy paid post	USDC transfer/split; grant buyer content access only after authoritative confirmation
JOB_FUND / PAIFLOW_JOB_FUND_WORKFLOW_ID	Client funds a job	Fund Soroban escrow if integration supports it; never bypass contract-controlled release

All Paiflow credentials and workflow IDs are server-only environment variables. Research deployments and user authorization; do not trust webhook alone as proof of payment. Validate chains, asset IDs, parties, amounts, fees, memos and replay protection.

04 / Features and screens

• Landing: responsive 3D looping hero, how-it-works, benefits, supported assets, trust and risk FAQs, footer and clear Try Senda CTA.

• Signup: Privy email OTP, map verified Privy subject to user profile, generate or retrieve compatible Stellar signing wallet, fund test account, select username.

• Social: following feed, profile, rich/media posts, likes, comments, reposts, notifications, moderation and creator tips.

• Wallet: balances from Stellar, send/receive by address or resolved handle, QR, fees, transaction timeline and network explorer links.

• Creator commerce: optional paid content with price in USDC, purchase and split, authenticated entitlement and secured file delivery.

• Jobs: create role/budget/deadline, apply or send offer, accept, funded escrow, in-progress chat, upload evidence, request revisions, approve/release, dispute/refund.

• Earn: clearly opt-in USDC Auto Earn, deposit and withdraw, underlying strategy, risks and changing displayed APY; no “wallet-only” magic yield.

• Faucet: signup funding and limited repeat claims on testnet only; show claim tx and next eligibility.

05 / Job lifecycle and contract custody

Job: DRAFT -> OFFERED -> ACCEPTED -> AWAITING_FUNDING -> FUNDED -> IN_PROGRESS -> PROOF_SUBMITTED -> (REVISION_REQUESTED | APPROVED | DISPUTED) -> PAID, or authorized REFUNDED. Each step has an allowed actor and recorded immutable event. Payment is locked in escrow before work commences; server status is a cache, never the money authority. Dispute outcomes require defined authorized mediator or agreed resolution rules, with abuse-resistant timeouts and audit trails.

Chat is per-job, authenticated, real-time, with attachments and evidence snapshots. Avoid “Done” automatically paying unless contract authorization and conditions are satisfied. Payment must be one-time and irreversible only at authorized release.

06 / USDC Hold & Earn

Display 5% APR as an ILLUSTRATIVE TARGET, not a promise. Example: 1,000 USDC at simple 5% for 365 days = 50 USDC before costs and losses, if that rate were maintained. Holding plain Stellar USDC in a wallet earns no interest. The user must opt in to deposit/allocate USDC into an identified, funded, audited yield source. Research lending/yield protocols such as Stellar Blend; do not assume there is an available 5%-APR pool.

Build an adapter/vault contract only with accurate share accounting, realized-yield reporting, withdrawal risk disclosures, limits, pause/guardian policies, strategy addresses and third-party risk. If no validated strategy is available, leave the production feature DISABLED and make testnet mode visibly experimental. No yield paid from other new-user deposits.

07 / Visual identity and mobile rules

Element	Final specification
Brand	Senda | Share more. Send easier. Earn together.
Palette	Ink #101B2D; cobalt #4169F5; cloud #F5F7FC; mint #24CBA8; white #FFFFFF
Typography	Headlines: Space Grotesk 600/700; body: Manrope 400/500/600; 14-16px minimum body; tabular financial numerals
Landing hero	Clear headline and concise supporting text, blue CTA Try Senda, interactive low-poly/glass 3D looping connection/ribbon motif
Motion	Slow seamless 8-16 second loops; never block scroll; prefers-reduced-motion static poster; low-power fallback
Desktop	12-column container ~1200px, left app sidebar, contextual right panel as available
Mobile	320-430px first; bottom 5-item navigation, no horizontal overflow, 44px targets, thumb-reachable CTAs
Accessibility	WCAG AA contrast, focus indicators, semantic elements, alt text, accessible modals and keyboard controls

Recommended implementation: Next.js App Router, Tailwind CSS, Framer Motion for microinteraction; Three.js/React Three Fiber for one optimized lazily loaded hero composition, SSR-safe with reduced-motion and static image fallbacks. Avoid massive moving backgrounds that harm mobile performance.

08 / Architecture and Supabase

Frontend Next.js/TypeScript + Tailwind; auth via Privy; blockchain via Stellar JS SDK / Soroban Rust; payment workflows server-side through verified Paiflow SDK/API; database Supabase PostgreSQL + storage + realtime; hosting Vercel; background reconciler and transaction indexer.

Supabase tables: profiles, linked_wallets, follows, posts, post_media, comments, tips, products, purchases, entitlements, jobs, applications, job_offers, conversations, messages, attachments, escrows, transfers, workflow_runs, earn_positions, faucet_claims, notifications, audit_logs, moderation_reports. Enforce RLS and verified subject mapping; use signed private storage URLs; unique event IDs, transaction indexes and reconciliation workers.

09 / Smart contract boundaries and layered security

• Rust/Soroban project with job_escrow, yield_vault, treasury_controls (only if required), shared interfaces and integration tests.

• Escrow invariants: correct token, exact deposit, authorized funder/recipient, immutable payout rules, nonnegative arithmetic, strict state machine, no double release, one payment per job, locked capital accounted for.

• Threat cases: replay, forged webhooks, impersonated usernames, malicious token, wrong network, duplicate funding, race conditions, reentrancy/cross-contract assumptions, expired ledger TTL, storage bloat, malicious upgrades.

• Privilege separation: user signs user movement; platform cannot unilaterally withdraw escrow; governor/multisig for upgrades; guardian pausing only as designed, emergency withdrawals constrained.

• Security features: CSRF/CSP/secure cookies, Privy session verification, rate limiting, encrypted secrets, audit logs, anomaly monitoring, abuse reports, upload scanning, independent contract audit before material mainnet funds.

10 / Deployer folder and secrets policy

Generate ops/deployer/ with README.md, .env.example, .gitignore, scripts/deploy-testnet.sh, scripts/verify.sh, manifests/. Keep deployer seed/private key OUTSIDE repo, PDF, web code and logs. Prefer managed secrets or hardware-backed signer; separate testnet/mainnet identities. Sample configs contain names/placeholder only. Never generate a plaintext secrets.txt, never ask users to paste a seed phrase in chat. Mainnet deploy/upgrades/funding require explicit human approval after checks.

Manifest must record PUBLIC account, network passphrase, contract address, WASM SHA-256, transaction hash, source commit, deployment time and permissions. Testnet deploy scripts can be executable; mainnet deployment must stay gated.

11 / Required files and environment variables

Files in delivery package: CLAUDE_MASTER_PROMPT.md, ENV_TEMPLATE.env.example and DEPLOYER_SECURITY_README.md. The env template defines PAIFLOW_SEND/RECEIVE/TIP/POST_PURCHASE/JOB_FUND_WORKFLOW_ID, API credentials, verified webhook secret, Stellar RPC/network/USDC identity, contract addresses, Supabase, Privy and faucet caps. No private key included.

12 / Milestones and acceptance tests

• Research and integration matrix: real documented endpoints, auth and network compatibility with links.

• Signup -> real funded testnet wallet -> functional send/receive XLM & USDC -> blockchain explorer confirmation.

• Tips and paid posts: correct split and entitlements exactly once across webhook retries.

• Jobs: full chat, proof, fund, approve, payout, revision, dispute and refund tests with adversarial cases.

• Earn: if enabled, yield accrues from proven strategy; else honestly disabled and testnet experiment labeled.

• Responsiveness: check 320, 375, 768, 1024, 1440px; no clipping; accessible keyboard/reduced-motion support.

• Unit, integration, property-based Soroban, E2E and security tests; transaction evidence and deployment manifests.

13 / Source documentation for Claude

https://github.com/artisam-paiflow/paiflow

https://github.com/artisam-paiflow/paiflow/blob/develop/docs/api/README.md

https://github.com/artisam-paiflow/paiflow/blob/develop/packages/sdk/README.md

https://beta.app.paiflow.xyz/

https://docs.privy.io/llms.txt

https://developers.stellar.org/docs/tools/cli/cookbook/upload-deploy

https://docs.blend.capital/

https://supabase.com/docs

Important implementation rule: no simulated success labels for failed transactions; a testnet prototype is not a mainnet production service. Never claim any contract deployed, feature completed, or workflow supported without verifying it.